ZOMBIE-PROCESS

좀비 프로세스와 자식 회수

연관 기출 · 140-1-8

한눈에 이해

실행은 끝났지만 부모가 종료 결과를 가져가지 않아 작은 종료 기록이 남아 있는 상태입니다.

정의와 핵심 원리

좀비 프로세스는 종료했으나 부모가 wait 계열 호출로 종료 상태를 회수하지 않아 프로세스 테이블의 상태 정보가 남은 프로세스입니다. 계속 실행하는 프로세스나 부모를 잃은 고아 프로세스와 구분합니다.

• 자식이 종료하면 주소 공간 등 대부분의 자원은 해제됩니다. PID, 종료 상태, 자원 사용 통계는 부모가 확인할 때까지 남습니다. wait, waitpid, waitid로 자식의 상태 변화를 기다리거나 종료 결과를 회수할 수 있습니다.

• 자식 여러 개가 비슷한 시점에 종료할 수 있습니다. 부모는 SIGCHLD 알림 횟수에만 의존하지 않고, 예를 들어 WNOHANG 옵션으로 종료한 자식을 반복해 회수합니다. SIGCHLD를 명시적으로 무시하거나 SA_NOCLDWAIT를 설정할 때의 동작은 운영체제 문서를 확인합니다.

• 좀비 프로세스는 이미 실행을 끝냈습니다. kill을 보내기보다 종료 상태를 거두지 않는 부모의 코드를 고치거나 적절한 재시작 절차를 밟습니다. 컨테이너에서도 PID 1이 자식의 종료 결과를 회수하는지 확인합니다.

구조·비교·판단

구분	핵심 원리	적용·한계
좀비	자식은 종료했으나 부모가 결과를 회수하지 않음	프로세스 슬롯·PID 고갈 가능
고아	부모가 먼저 종료	다른 관리 주체가 부모 역할을 맡음
실행·대기	작업이 아직 끝나지 않음	CPU·입출력·잠금 상태 분석

적용 예제

배치 서버의 종료 누락

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 부모는 매초 자식을 10개 생성합니다. 자식은 모두 즉시 종료하고 부모는 wait를 호출하지 않습니다.

1. ps에서 상태 Z와 부모 PID를 확인합니다.

2. 60초 동안 최대 600개의 미회수 종료 기록이 쌓일 수 있습니다.

3. 부모에 회수 루프를 추가해 회수할 수 있는 자식의 종료 결과를 모두 가져옵니다.

4. 부하를 반복한 뒤 Z 상태가 계속 쌓이지 않고 종료 코드도 정상적으로 처리되는지 확인합니다.

결과·해석: CPU 사용량보다 미회수 자식 수의 증가를 증거로 원인을 판단합니다.

전제·주의: 부모 강제 종료는 다른 업무를 중단할 수 있으므로 프로그램 수정과 정상 관리 절차를 우선합니다.

답안 골격과 혼동 방지

정의와 상태 전이 → 종료 후 남는 정보 → wait와 SIGCHLD → 고아 프로세스와 비교 → 컨테이너의 회수 방식

- 좀비의 실행은 끝났으며 종료 기록만 남습니다.
- 메모리 전체가 계속 점유된다고 설명하지 않습니다.

내용을 가리고 회상

1. 좀비 프로세스와 고아 프로세스는 어떻게 다릅니까?

2. wait가 회수하는 정보는 무엇입니까?

3. SIGCHLD를 한 번 받을 때 종료한 자식을 하나만 회수하면 왜 부족합니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- wait(2) - Linux man-pages project; 상시 문서; 확인 2026-11-05.